# CN ORANGE PROBLEM

# Link Failure Detection and Recovery

Name : Sharanya R

SRN: PES1UG24CS902

Class: 4th Sem, Section A

**Problem Statement**

Detect link failures and update routing dynamically – In a computer network, when a link between two devices fails, the traffic attempts to transmit through the failed link, thus causing packet loss.

To prevent this, the traffic should be distributed dynamically to other working links to ensure that the data reaches the destination without packet loss.

The objective is to create an SDN based solution which

- Monitors topology changes in real time
- Detects link failure in the automatically
- Updates flow rules dynamically to reroute the traffic
- Restores connectivity without manual intervention

There are two topologies used here –

- Linear – this is a simple topology with one link between the switches to demonstrate how the traffic flows between the hosts when there is a path. This has no alternate path, hence when the link between the switches s1 and s2 is down, the packets are lost.
- Triangular – this topology has 3 switches – s1,s2,s3. Host h1 has a link to s1, and host h3 has a link to s3. The switches are interconnected with each other to form a triangle. When one link is down, the traffic is rerouted through other links, and the pingall is a success. A triangular topology provides multiple independent paths between any two nodes, enabling fault tolerance. This topology is the simplest to demonstrate rerouting, and hence was chosen. The graph contains a cycle, so, removing one edge does NOT disconnect the graph – there is always an alternate path between nodes.

**Setup Steps**

- Install mininet using

```
sudo apt update
sudo apt upgrade -y
sudo apt install mininet -y
```

- Install ryu using

```
pip install ryu
```

OR

Run the following commands:

```
sudo apt update
sudo apt install -y git python3 python3-pip python3-dev
git clone https://github.com/faucetsdn/ryu.git
cd ryu
pip3 install -e .
```

- Cd into the directory with the codes
- In one terminal, start the ryu controller. This should remain running in the background

```
ryu-manager controller.py
```

- In another terminal, open the mininet cli using the command

```
sudo python3 topology.py
```

Enter the commands in the mininet cli

**Proof of Execution**

LINEAR TOPOLOGY –

```
sharanya@sharanya-VirtualBox:~/sdn_mininet$ tail -f controller_logs.txt
[SWITCH_CONNECT] Switch DPID=0x1 connected
[TOPOLOGY] Switch 0x1 entered
[SWITCH_CONNECT] Switch DPID=0x2 connected
[TOPOLOGY] Switch 0x2 entered
[MAC_LEARN] DPID=0x1 MAC 8e:9f:67:f7:c3:bb on port 1
[MAC_LEARN] DPID=0x2 MAC 46:cb:82:86:92:de on port 1
[MAC_LEARN] DPID=0x1 MAC 46:cb:82:86:92:de on port 2
[MAC_LEARN] DPID=0x2 MAC 8e:9f:67:f7:c3:bb on port 2
[MAC_LEARN] DPID=0x1 MAC da:3f:25:5b:bd:5b on port 2
[MAC_LEARN] DPID=0x2 MAC ba:e6:8d:70:66:7b on port 2
```

Controller output, shows how the paths are being learnt

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=4.385s, table=0, n_packets=23, n_bytes=2703, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=10.277s, table=0, n_packets=30, n_bytes=3430, priority=0 actions=CONTROLLER:65535
```

In the very beginning, the flow rules have not been installed in the switches, hence packets of the new flow are sent to the controller via Packet-In messages.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

When we do pingall, we can see that all the packets have transmitted successfully, showing that the link is working well.

```
mininet> h1 ping -c 10 -i 0.5 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.508 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.043 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.041 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.054 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.047 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.041 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.040 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.043 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 4538ms
rtt min/avg/max/mdev = 0.040/0.092/0.508/0.138 ms
mininet> h2 ping -c 10 -i 0.5 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.141 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.041 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.041 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.195 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.040 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.042 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=0.043 ms
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=0.205 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=0.060 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=0.056 ms

--- 10.0.0.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 4669ms
rtt min/avg/max/mdev = 0.040/0.086/0.205/0.063 ms
```

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=18.337s, table=0, n_packets=23, n_bytes=2198, idle_timeout=15, priority=10,in_port="s1-eth2",dl_src=46:cb:82:86:92
:de,dl_dst=8e:9f:67:f7:c3:bb actions=output:"s1-eth1"
 cookie=0x0, duration=18.332s, table=0, n_packets=22, n_bytes=2100, idle_timeout=15, priority=10,in_port="s1-eth1",dl_src=8e:9f:67:f7:c3
:bb,dl_dst=46:cb:82:86:92:de actions=output:"s1-eth2"
 cookie=0x0, duration=34.190s, table=0, n_packets=40, n_bytes=4156, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=21.186s, table=0, n_packets=23, n_bytes=2198, idle_timeout=15, priority=10,in_port="s2-eth1",dl_src=46:cb:82:86:92
:de,dl_dst=8e:9f:67:f7:c3:bb actions=output:"s2-eth2"
 cookie=0x0, duration=21.179s, table=0, n_packets=22, n_bytes=2100, idle_timeout=15, priority=10,in_port="s2-eth2",dl_src=8e:9f:67:f7:c3
:bb,dl_dst=46:cb:82:86:92:de actions=output:"s2-eth1"
 cookie=0x0, duration=37.007s, table=0, n_packets=41, n_bytes=4246, priority=0 actions=CONTROLLER:65535
```

After the ping has been performed, we can see that the switches learn the flow rules, and the full flow table is displayed.

IPERF output

```
mininet> h2 iperf -s -i 1 &
mininet> h1 iperf -c 10.0.0.2 -t 10 -i 2
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 56108 connected with 10.0.0.2 port 5001 (icwnd/mss/irtt=14/1448/35)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-2.0000 sec  13.0 GBytes  55.7 Gbits/sec
[  1] 2.0000-4.0000 sec  12.0 GBytes  51.6 Gbits/sec
[  1] 4.0000-6.0000 sec  4.83 GBytes  20.8 Gbits/sec
[  1] 6.0000-8.0000 sec  10.1 GBytes  43.3 Gbits/sec
[  1] 8.0000-10.0000 sec  12.1 GBytes  52.0 Gbits/sec
[  1] 0.0000-10.0079 sec  52.0 GBytes  44.6 Gbits/sec
mininet> h2 pkill iperf
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.2 port 5001 connected with 10.0.0.1 port 56108 (icwnd/mss/irtt=14/1448/18)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-1.0000 sec  6.47 GBytes  55.6 Gbits/sec
[  1] 1.0000-2.0000 sec  6.50 GBytes  55.8 Gbits/sec
[  1] 2.0000-3.0000 sec  6.61 GBytes  56.8 Gbits/sec
[  1] 3.0000-4.0000 sec  5.40 GBytes  46.4 Gbits/sec
[  1] 4.0000-5.0000 sec  2.16 GBytes  18.6 Gbits/sec
[  1] 5.0000-6.0000 sec  2.67 GBytes  22.9 Gbits/sec
[  1] 6.0000-7.0000 sec  4.17 GBytes  35.8 Gbits/sec
[  1] 7.0000-8.0000 sec  5.90 GBytes  50.7 Gbits/sec
[  1] 8.0000-9.0000 sec  6.17 GBytes  53.0 Gbits/sec
[  1] 9.0000-10.0000 sec  5.94 GBytes  51.0 Gbits/sec
[  1] 0.0000-10.0002 sec  52.0 GBytes  44.7 Gbits/sec
mininet> h2 pkill iperf
```

```
mininet> h2 pkill iperf
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=38.416s, table=0, n_packets=1193964, n_bytes=55899378404, idle_timeout=15, priority=10,in_port="s1-eth1",dl_src=8e
:9f:67:f7:c3:bb,dl_dst=46:cb:82:86:92:de actions=output:"s1-eth2"
 cookie=0x0, duration=38.413s, table=0, n_packets=434246, n_bytes=28661304, idle_timeout=15, priority=10,in_port="s1-eth2",dl_src=46:cb:
82:86:92:de,dl_dst=8e:9f:67:f7:c3:bb actions=output:"s1-eth1"
 cookie=0x0, duration=108.152s, table=0, n_packets=46, n_bytes=4669, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=110.940s, table=0, n_packets=47, n_bytes=4759, priority=0 actions=CONTROLLER:65535
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

We can observe that the number of packets in the flow table has increased drastically, because the iperf generates high throughput TCP traffic, resulting in a large number of data packets and ACK packets, which rapidly increases flow counters

Now, when the link is down, we can see that the packets dropped (100%). Only when the link is restored, the packets can be sent. This is because there is no alternate path through which the packets can be rerouted, hence no new flow rule can be installed for a non-existent link.

```
mininet> link s1 s2 down
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4124ms

mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=16.594s, table=0, n_packets=7, n_bytes=518, idle_timeout=15, priority=10,in_port="s1-eth1",dl_src=8e:9f:67:f7:c3:b
b,dl_dst=46:cb:82:86:92:de actions=output:"s1-eth2"
 cookie=0x0, duration=157.438s, table=0, n_packets=51, n_bytes=5140, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=162.630s, table=0, n_packets=53, n_bytes=5272, priority=0 actions=CONTROLLER:65535
mininet> link s1 s2 up
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
s1 lo:  s1-eth1:h1-eth0 s1-eth2:s2-eth2
s2 lo:  s2-eth1:h2-eth0 s2-eth2:s1-eth2
c0
```

This is the wireshark output-

```
sharanya@sharanya-VirtualBox:~/sdn_mininet$ sudo tcpdump -i s1-eth2 -c 50 -w ping_capture.pcap -vv 2>&1 | tee ping_wireshark.log
tcpdump: listening on s1-eth2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
50 packets captured
78 packets received by filter
0 packets dropped by kernel
sharanya@sharanya-VirtualBox:~/sdn_mininet$ sudo tcpdump -i s1-eth2 -c 50 -w ping_capture.pcap -vv 2>&1 | tee ping_wireshark.log
tcpdump: listening on s1-eth2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
50 packets captured
50 packets received by filter
0 packets dropped by kernel
```

TRIANGULAR TOPOLOGY –

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=7.232s, table=0, n_packets=8715, n_bytes=921220, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=8.639s, table=0, n_packets=11414, n_bytes=1198849, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s3
 cookie=0x0, duration=10.009s, table=0, n_packets=13261, n_bytes=1387253, priority=0 actions=CONTROLLER:65535
```

Flow table has no installed rules.

Ping output -

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

```
mininet> h1 ping -c 10 -i 0.5 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.037 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.032 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.028 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.988 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.048 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.044 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.028 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.035 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.027 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 4527ms
rtt min/avg/max/mdev = 0.027/0.133/0.988/0.285 ms
mininet> h2 ping -c 10 -i 0.5 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.028 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.023 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.379 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.041 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.050 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.047 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=0.050 ms
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=0.028 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=0.042 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=0.030 ms

--- 10.0.0.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 4551ms
rtt min/avg/max/mdev = 0.023/0.071/0.379/0.102 ms
```

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=15.140s, table=0, n_packets=88, n_bytes=4928, idle_timeout=15, priority=10,in_port="s1-eth3",dl_src=d6:38:d5:74:74
:ef,dl_dst=7e:2e:b7:86:6e:71 actions=output:"s1-eth1"
 cookie=0x0, duration=15.051s, table=0, n_packets=22, n_bytes=2100, idle_timeout=15, priority=10,in_port="s1-eth1",dl_src=7e:2e:b7:86:6e
:71,dl_dst=d6:38:d5:74:74:ef actions=output:"s1-eth3"
 cookie=0x0, duration=50.282s, table=0, n_packets=72589, n_bytes=7023705, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s3
 cookie=0x0, duration=17.789s, table=0, n_packets=100, n_bytes=5432, idle_timeout=15, priority=10,in_port="s3-eth1",dl_src=d6:38:d5:74:7
4:ef,dl_dst=7e:2e:b7:86:6e:71 actions=output:"s3-eth3"
 cookie=0x0, duration=17.424s, table=0, n_packets=22, n_bytes=2100, idle_timeout=15, priority=10,in_port="s3-eth3",dl_src=7e:2e:b7:86:6e
:71,dl_dst=d6:38:d5:74:74:ef actions=output:"s3-eth1"
 cookie=0x0, duration=52.701s, table=0, n_packets=77774, n_bytes=7501594, priority=0 actions=CONTROLLER:65535
```

Rules have been installed in the flow table.

IPERF Results –

```
mininet> h2 iperf -s -i 1 &
mininet> h1 iperf -c 10.0.0.2 -t 10 -i 2
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 38910 connected with 10.0.0.2 port 5001 (icwnd/mss/irtt=14/1448/479276)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-2.0000 sec  3.11 GBytes  13.3 Gbits/sec
[  1] 2.0000-4.0000 sec  2.76 GBytes  11.9 Gbits/sec
[  1] 4.0000-6.0000 sec  4.99 GBytes  21.5 Gbits/sec
[  1] 6.0000-8.0000 sec  5.79 GBytes  24.9 Gbits/sec
[  1] 8.0000-10.0000 sec  2.66 GBytes  11.4 Gbits/sec
[  1] 0.0000-10.1112 sec  19.3 GBytes  16.4 Gbits/sec
mininet> h2 pkill iperf
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.2 port 5001 connected with 10.0.0.1 port 38910 (icwnd/mss/irtt=14/1448/187871)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-1.0000 sec  1.66 GBytes  14.2 Gbits/sec
[  1] 1.0000-2.0000 sec  1.66 GBytes  14.2 Gbits/sec
[  1] 2.0000-3.0000 sec  1.28 GBytes  11.0 Gbits/sec
[  1] 3.0000-4.0000 sec  1.45 GBytes  12.5 Gbits/sec
[  1] 4.0000-5.0000 sec  2.30 GBytes  19.8 Gbits/sec
[  1] 5.0000-6.0000 sec  3.05 GBytes  26.2 Gbits/sec
[  1] 6.0000-7.0000 sec  2.82 GBytes  24.3 Gbits/sec
[  1] 7.0000-8.0000 sec  3.00 GBytes  25.8 Gbits/sec
[  1] 8.0000-9.0000 sec  1.49 GBytes  12.8 Gbits/sec
[  1] 9.0000-9.8134 sec   625 MBytes  6.44 Gbits/sec
[  1] 0.0000-9.8134 sec  19.3 GBytes  16.9 Gbits/sec
```

```
mininet> link s1 s2 down
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=5.59 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.552 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.037 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.039 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.053 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4105ms
```

Now, link s1 s2 is down. So, packets are rerouted via link s1 s3, and still reach the destination successfully, hence ping shows 0% packet loss. Thus link failure is detected automatically, flow rules are updated dynamically and connectivity is restored.

- RYU topology module detects link removal, which is automatically triggered when a link goes down.
- The controller removes the link from the active topology, marks the ports as failed, and triggers recovery.
- Recovery mechanism – Flow rules are cleared, but not the MAC table entries, because the destination needs to be known. Then, the forwarding decision is taken.
- Alternate routing takes place – the destination switch is marked, and the shortest path is computed  using dijkstra's algorithm, and the next hop port is returned.
- EventLinkAdd is used for link restoration – when a link becomes up, it is added to the active_links, and removed from down_links. The flows are updated implicitly on next packet. This is an event-driven mechanism.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=144.081s, table=0, n_packets=393507, n_bytes=25971170, idle_timeout=15, priority=10,in_port="s1-eth3",dl_src=d6:38
:d5:74:74:ef,dl_dst=7e:2e:b7:86:6e:71 actions=output:"s1-eth1"
 cookie=0x0, duration=8.823s, table=0, n_packets=5, n_bytes=434, idle_timeout=15, priority=10,in_port="s1-eth1",dl_src=7e:2e:b7:86:6e:71
,dl_dst=d6:38:d5:74:74:ef actions=output:"s1-eth3"
 cookie=0x0, duration=179.223s, table=0, n_packets=268712, n_bytes=24749212, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s3
 cookie=0x0, duration=147.029s, table=0, n_packets=393506, n_bytes=25971128, idle_timeout=15, priority=10,in_port="s3-eth1",dl_src=d6:38
:d5:74:74:ef,dl_dst=7e:2e:b7:86:6e:71 actions=output:"s3-eth3"
 cookie=0x0, duration=11.609s, table=0, n_packets=5, n_bytes=434, idle_timeout=15, priority=10,in_port="s3-eth3",dl_src=7e:2e:b7:86:6e:7
1,dl_dst=d6:38:d5:74:74:ef actions=output:"s3-eth1"
 cookie=0x0, duration=181.941s, table=0, n_packets=268712, n_bytes=24749164, priority=0 actions=CONTROLLER:65535
mininet> link s1 s2 up
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s3-eth1
s1 lo:  s1-eth1:h1-eth0 s1-eth2:s2-eth1 s1-eth3:s3-eth3
s2 lo:  s2-eth1:s1-eth2 s2-eth2:s3-eth2
s3 lo:  s3-eth1:h2-eth0 s3-eth2:s2-eth2 s3-eth3:s1-eth3
c0
```

WIRESHARK output –

```
sharanya@sharanya-VirtualBox:~/sdn_mininet$ sudo tcpdump -i s1-eth2 -c 50 -w ping_capture.pcap -vv 2>&1 | tee ping_wireshark.log
tcpdump: listening on s1-eth2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
50 packets captured
1447 packets received by filter
0 packets dropped by kernel
sharanya@sharanya-VirtualBox:~/sdn_mininet$ sudo tcpdump -i s1-eth2 -c 100 -w link_failure_capture.pcap -vv 2>&1 | tee link_failure_wi
shark.log
tcpdump: listening on s1-eth2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
100 packets captured
1463 packets received by filter
0 packets dropped by kernel
```